Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

Методы машинного обучения

ОТЧЕТ

по лабораторной работе №1

Вариант 21

Выполнил
Студент гр. 351002 Михальков А. Д.

Проверил Лебедевич А. В.

Минск 2026

СОДЕРЖАНИЕ

1 Цель выполнения 3
2 Теоретическая основа 3
2.1 Базовые структуры данных и коллекции в Python 3
2.2 Обработка исключительных ситуаций 3
2.3 Функции и декомпозиция кода 4
2.4 Объектно-ориентированное программирование 4
2.5 Типы методов в классах 4
2.6 Наследование и полиморфизм 5
2.7 Работа с файловой системой 5
3 Индивидуальное задание 6
3.1 Базовые конструкции и структуры данных 6
3.2 Функции и обработка исключительных ситуаций 6
3.3 Объектно-ориентированное программирование 7
4 Код решения 8
4.1 Код решения Блока 1 8
4.2 Код решения Блока 2 10
4.3 Код решения Блока 3 13
5 Скриншоты выполнения 18
6 Выводы 22

1 Цель выполнения

Овладение практическими навыками разработки программного обеспечения на языке высокого уровня Python. Изучение и применение на практике базовых структур данных (строк, списков, кортежей, словарей), реализация алгоритмов с использованием функций и организация отказоустойчивого кода посредством обработки исключительных ситуаций (блок try-except-finally). Освоение принципов объектно-ориентированного программирования (ООП), включая проектирование классов, работу со статическими и динамическими переменными, применение наследования и полиморфизма методов (экземпляра, класса и статических), а также реализация механизмов взаимодействия с внешней файловой системой.

2 Теоретическая основа

2.1 Базовые структуры данных и коллекции в Python

Язык программирования Python предоставляет мощный набор встроенных коллекций для хранения данных и манипулирования ими:

– Списки (list) – это упорядоченные изменяемые коллекции объектов произвольных типов. Доступ к элементам осуществляется по индексу. Списки поддерживают срезы (slices), позволяющие извлекать части последовательности.

– Кортежи (tuple) – это упорядоченные, но неизменяемые коллекции. Использование кортежей повышает безопасность данных, предотвращая их случайное изменение, и работает быстрее списков в плане потребления ресурсов памяти.

– Словари (dict) – это неупорядоченные (начиная с Python 3.7 – сохраняющие порядок вставки) коллекции пар «ключ – значение». Доступ к данным осуществляется по уникальному ключу, что обеспечивает высокую скорость поиска.

2.2 Обработка исключительных ситуаций

Исключения в Python – это инструмент для обработки ошибок, возникающих во время выполнения программы. Для управления ошибками используется конструкция try-except-else-finally:

– try – содержит программный код, в котором может возникнуть исключительная ситуация.

– except – блок кода, выполняемый при возникновении ошибки определенного типа (например, ValueError при неверном типе данных или ZeroDivisionError при делении на ноль).

– else – выполняется в том случае, если в блоке try не возникло исключений.

– finally – блок, код в котором выполняется в любом случае (возникла ошибка или нет). Обычно используется для освобождения внешних ресурсов (закрытие файлов, соединений).

2.3 Функции и декомпозиция кода

Функция в Python – это именованный блок кода, который выполняется только при вызове. Функции позволяют повторно использовать код и структурировать логику программы. Ключевое слово def используется для определения функции. Документирование функций осуществляется с помощью строк документации (docstrings), обрамленных тройными кавычками.

2.4 Объектно-ориентированное программирование

ООП – это методология, основанная на представлении программы в виде совокупности объектов.

Класс – это «чертеж» или шаблон для создания объектов.

Объект (экземпляр) – конкретное воплощение класса в памяти компьютера.

Переменные класса (статические) – переменные, определенные внутри класса, но вне методов. Они являются общими для всех экземпляров данного класса.

Переменные экземпляра (динамические) – переменные, создаваемые внутри методов (обычно в конструкторе) с приставкой self. Они уникальны для каждого конкретного объекта.

2.5 Типы методов в классах

В Python выделяют три основных типа методов:

– Методы экземпляра – принимают в качестве первого аргумента self (ссылку на объект). Используются для работы с атрибутами конкретного объекта.

– Методы класса (@classmethod) – принимают в качестве аргумента cls (ссылку на сам класс). Используются для управления данными, общими для всех объектов, или как альтернативные конструкторы.

– Статические методы (@staticmethod) – не принимают ссылок на объект или класс. Это обычные функции, помещенные в пространство имен класса для логической группировки.

2.6 Наследование и полиморфизм

Наследование позволяет создать новый класс на основе уже существующего, перенимая его атрибуты и методы. Это способствует повторному использованию кода и созданию иерархий.

Полиморфизм позволяет объектам разных классов (например, «Рыбы» и «Птицы») иметь методы с одинаковыми именами, но различной реализацией (переопределение методов).

2.7 Работа с файловой системой

Python предоставляет встроенные средства для работы с файлами. Рекомендуемым способом открытия файлов является менеджер контекста with open(...) as f:. Данная конструкция гарантирует корректное закрытие файла после завершения операций, даже если в процессе возникла ошибка. Основные режимы работы: "r" (чтение), "w" (запись с перезаписью), "a" (добавление в конец).

3 Индивидуальное задание

В соответствии с вариантом, индивидуальное задание разделено на три логических блока: базовые структуры данных, использование функций с обработкой исключений и объектно-ориентированное программирование.

3.1 Базовые конструкции и структуры данных

1 Разработать программу, которая выводит на экран все делители числа, введенного с клавиатуры.

2 Реализовать обработку строковых данных:

2.1 подсчитать количество слов, имеющих четную длину;

2.2 найти и вывести на экран самое длинное слово в строке.

3 Выполнить операции над списком чисел:

3.1 найти наибольший четный элемент (если такового нет – вывести первый элемент);

3.2 преобразовать список так, чтобы сначала шли положительные элементы, а затем все остальные.

4 В заданном словаре my_dict найти три ключа с самыми маленькими значениями.

5 Реализовать интерактивную программу «Ювелирный магазин» с использованием словаря. Программа должна включать меню: просмотр описания, цены, количества, вывод всей информации, покупка товара (с обновлением остатков) и выход.

6 Принять от пользователя последовательность чисел через запятую и сформировать на их основе список и кортеж.

3.2 Функции и обработка исключительных ситуаций

1 Написать функцию для определения делителей числа, включив в неё обработку ошибок ввода данных с помощью конструкции try-except-finally.

2 Разработать универсальную функцию, принимающую один аргумент и выполняющую действия в зависимости от его типа:

2.1 для списка: расчет суммы между двумя первыми нулями;

2.2 для кортежа: поиск min/max элементов и их обмен местами;

2.3 для числа: расчет суммы четных цифр;

2.4 для строки: преобразование каждого символа в код Unicode.

3 Реализовать поиск в матрице первой строки, элементы которой упорядочены по возрастанию, и изменить порядок её элементов на обратный.

4 Написать программу, демонстрирующую пошаговую работу механизмов try, except, else и finally при выполнении арифметических операций.

3.3 Объектно-ориентированное программирование

1 Создать класс Example, содержащий статические и динамические переменные, а также три метода: вывод локальной переменной, возврат суммы глобальных переменных и возведение одной динамической переменной в степень другой.

2 Реализовать класс Employee (Сотрудник), включающий поля: ФИО, стаж, часовая ставка, отработанные часы. Реализовать методы расчета оклада и премии (процент премии зависит от стажа: до 1 года – 1%, до 3 лет – 5%, до 5 лет – 8%, свыше 5 лет – 15%).

3 Спроектировать иерархию классов «Зоомагазин» – «Животное» – «Рыбы»/«Птицы». Реализовать свойства (порода, стоимость), переопределение методов передвижения, поиск самого дорогого товара и метод записи инвентаризационных данных в текстовый файл.

4 Разработать собственный класс, демонстрирующий практическое применение различных типов методов: экземпляра класса, методов класса (@classmethod) и статических методов (@staticmethod).

4 Код решения

4.1 Код решения Блока 1

```
# Блок 1, Задание 1
try:
    number = int(input("Введите целое число: "))
    print(f"Делители числа {number}:")

    for i in range(1, abs(number) + 1):
        if number % i == 0:
            print(i, end=" ")
except ValueError:
    print("Ошибка! Пожалуйста, введите целое число.")
# Блок 1, Задание 2
user_input = input("Введите предложение: ")

words = user_input.split()

even_count = 0
for word in words:
    clean_word = word.strip(".,!?;:-")
    if len(clean_word) % 2 == 0 and len(clean_word) > 0:
        even_count += 1

if words:
    longest_word = max(words, key=len)

    print(f"Количество слов с четной длиной: {even_count}")
    print(f"Самое длинное слово: {longest_word}")
else:
    print("Вы ввели пустую строку.")

# Блок 1, Задание 3
raw_input = input("Введите числа через пробел: ")
my_list = [int(x) for x in raw_input.split()]

if not my_list:
    print("Список пуст!")
else:
    evens = [x for x in my_list if x % 2 == 0]

    if evens:
        result_max = max(evens)
        print(f"Наибольший четный элемент: {result_max}")
    else:
        result_max = my_list[0]
        print(f"Четных элементов нет. Первый элемент списка: {result_max}")

    positive_part = [x for x in my_list if x > 0]
    others_part = [x for x in my_list if x <= 0]

    new_list = positive_part + others_part
    print(f"Преобразованный список: {new_list}")

# Блок 1, Задание 4
```

```
my_dict = {'a': 500, 'b': 5874, 'c': 560, 'd': 400, 'e': 5874, 'f': 20}

sorted_keys = sorted(my_dict, key=my_dict.get)

result_keys = sorted_keys[:3]

print(f"Исходный словарь: {my_dict}")
print(f"Три ключа с самыми маленькими значениями: {result_keys}")

for key in result_keys:
    print(f"Ключ '{key}', значение: {my_dict[key]}")

# Блок 1, Задание 5
items = {
    "Кольцо": ["Золото", 15000, 10],
    "Серьги": ["Серебро", 5000, 20],
    "Подвеска": ["Платина", 25000, 5],
    "Браслет": ["Золото", 12000, 8]
}

while True:
    print("\n--- МЕНЮ МАГАЗИНА ---")
    print("1. Просмотр описания (название – состав)")
    print("2. Просмотр цены (название – цена)")
    print("3. Просмотр количества (название – количество)")
    print("4. Вся информация")
    print("5. Покупка")
    print("6. До свидания")

    choice = input("\nВыберите пункт меню: ")

    if choice == '1':
        for name, info in items.items():
            print(f"{name} – {info[0]}")

    elif choice == '2':
        for name, info in items.items():
            print(f"{name} – {info[1]} руб.")

    elif choice == '3':
        for name, info in items.items():
            print(f"{name} – {info[2]} шт.")

    elif choice == '4':
        print(f"{'Название':<12} | {'Состав':<10} | {'Цена':<8} | {'Кол-во'}")
        print("-" * 45)
        for name, info in items.items():
            print(f"{name:<12} | {info[0]:<10} | {info[1]:<8} | {info[2]}")

    elif choice == '5':
        print("Введите 'n' для выхода из режима покупки.")
        total_sum = 0

        while True:
            buy_item = input("\nВведите название изделия: ")
            if buy_item.lower() == 'n':
                break
```

```
            if buy_item in items:
                try:
                    count = int(input(f"Сколько штук '{buy_item}' хотите купить? "))

                    if count <= items[buy_item][2]:
                        cost = count * items[buy_item][1]
                        total_sum += cost

                        items[buy_item][2] -= count

                        print(f"Добавлено в чек. Сумма за {count} шт: {cost}")
                        print(f"Осталось в магазине: {items[buy_item][2]} шт.")
                    else:
                        print("Недостаточно товара на складе!")
                except ValueError:
                    print("Ошибка! Введите числовое количество.")
            else:
                print("Такого товара нет в ассортименте.")

        print(f"\nИТОГО К ОПЛАТЕ: {total_sum} руб.")

    elif choice == '6':
        print("До свидания! Приходите еще.")
        break
    else:
        print("Неверный пункт меню, попробуйте снова.")

# Блок 1, Задание 6
user_data = input("Введите числа через запятую (например: 1 2 3 4): ")

numbers_list = [int(x.strip()) for x in user_data.split()]

numbers_tuple = tuple(numbers_list)

print(f"Список (List): {numbers_list}")
print(f"Кортеж (Tuple): {numbers_tuple}")

print(f"Тип первой переменной: {type(numbers_list)}")
print(f"Тип второй переменной: {type(numbers_tuple)}")
```

4.2 Код решения Блока 2

```
# Блок 2, Задание 1
def find_divisors(n):
    """Функция возвращает список делителей числа n."""
    divisors = []
    for i in range(1, abs(n) + 1):
        if n % i == 0:
            divisors.append(i)
    return divisors

def main_task_1():
    try:
        user_input = input("Введите целое число для поиска делителей: ")
        number = int(user_input)
```

```
        result = find_divisors(number)

        print(f"Делители числа {number}: {result}")

    except ValueError:
        print("Ошибка: Введено не целое число! Пожалуйста, используйте только
цифры.")
    except Exception as e:
        print(f"Произошла непредвиденная ошибка: {e}")
    finally:
        print("Расчет делителей завершен.")

main_task_1()

# Блок 2, Задание 2
def multi_functional_handler(data):
    print(f"\n--- Обработка типа: {type(data).__name__} ---")
    try:
        if isinstance(data, list):
            first_zero = data.index(0)
            second_zero = data.index(0, first_zero + 1)
            result = sum(data[first_zero + 1 : second_zero])
            print(f"Сумма между нулями: {result}")
            return result

        elif isinstance(data, tuple):
            temp_list = list(data)
            idx_max = temp_list.index(max(temp_list))
            idx_min = temp_list.index(min(temp_list))
            temp_list[idx_max], temp_list[idx_min] = temp_list[idx_min],
temp_list[idx_max]
            result = tuple(temp_list)
            print(f"Кортеж после обмена min и max: {result}")
            return result

        elif isinstance(data, (int, float)):
            digits = [int(d) for d in str(abs(int(data))) if d.isdigit()]
            even_sum = sum(d for d in digits if d % 2 == 0)
            print(f"Сумма четных цифр числа: {even_sum}")
            return even_sum

        elif isinstance(data, str):
            unicode_codes = [ord(char) for char in data]
            print(f"Коды Unicode: {unicode_codes}")
            return unicode_codes

        else:
            print("Передан неизвестный тип данных.")

    except ValueError:
        print("Ошибка: В данных нет нужного элемента (например, не хватает нулей в
списке) или данные пусты.")
    except Exception as e:
        print(f"Произошла ошибка: {e}")
    finally:
        print("Завершение обработки аргумента.")

try:
```

```
    multi_functional_handler([1, 0, 5, 5, 0, 10])
    multi_functional_handler((10, 2, 55, 1, 8))
    multi_functional_handler(123456)
    multi_functional_handler("Python")
except Exception as e:
    print(f"Ошибка при вызове проверок: {e}")

# Блок 2, Задание 3
def reverse_first_sorted_row(matrix):
    """Ищет первую возрастающую строку и разворачивает её."""
    found = False

    for i in range(len(matrix)):
        row = matrix[i]
        if row == sorted(row):
            print(f"Найдена подходящая строка на индексе {i}: {row}")
            matrix[i] = row[::-1]
            found = True
            break

    if not found:
        print("В матрице нет строк, упорядоченных по возрастанию.")

    return matrix

def main_task_3():
    print("--- Запуск задания 3 (Матрицы) ---")

    test_matrix = [
        [5, 8, 2],
        [1, 2, 3],
        [4, 5, 6]
    ]

    try:
        print("Исходная матрица:")
        for r in test_matrix: print(r)

        result_matrix = reverse_first_sorted_row(test_matrix)

        print("\nМатрица после обработки:")
        for r in result_matrix: print(r)

    except TypeError:
        print("Ошибка: В матрице должны быть только числа.")
    except Exception as e:
        print(f"Произошла ошибка: {e}")
    finally:
        print("\nРабота с матрицей завершена.")

main_task_3()

# Блок 2, Задание 4
def demonstrate_exception_handling():
    print("--- Демонстрация работы try...except...finally ---")

    try:
        num1 = input("Введите делимое (число): ")
```

```
        num2 = input("Введите делитель (число): ")

        result = float(num1) / float(num2)

    except ValueError:
        print("ИСКЛЮЧЕНИЕ: Ошибка значения! Вы ввели не число.")
        result = None

    except ZeroDivisionError:
        print("ИСКЛЮЧЕНИЕ: Ошибка деления на ноль! На 0 делить нельзя.")
        result = None

    except Exception as e:
        print(f"ИСКЛЮЧЕНИЕ: Произошло что-то странное: {e}")
        result = None

    else:
        print(f"УСПЕХ: Результат деления равен {result}")

    finally:
        print("БЛОК FINALLY: Работа функции завершена, ресурсы освобождены.")
        return result

print("Тест 1:")
demonstrate_exception_handling()

print("\nТест 2:")
demonstrate_exception_handling()

print("\nТест 3:")
demonstrate_exception_handling()
```

4.3 Код решения Блока 3

```
# Блок 3, Задание 1
global_var1 = 10
global_var2 = 5

class Example:
    a = 100
    b = 200

    def __init__(self, dyn1, dyn2):
        self.dyn1 = dyn1
        self.dyn2 = dyn2

    def method1(self):
        local_v = "Я локальная переменная внутри метода 1"
        print(local_v)

    def method2(self):
        return global_var1 + global_var2

    def method3(self):
        return self.dyn1 ** self.dyn2
```

```
obj = Example(2, 3)

print("Вызов метода 1:")
obj.method1()

print(f"Результат метода 2 (сумма глобальных): {obj.method2()}")

print(f"Результат метода 3 (возведение в степень): {obj.method3()}")

print(f"Статическая переменная a = {obj.a}")


# Блок 3, Задание 2
class Employee:
    def __init__(self, fio, experience, hourly_wage, hours_worked):
        self.fio = fio
        self.experience = experience
        self.hourly_wage = hourly_wage
        self.hours_worked = hours_worked
        self.salary = 0
        self.bonus = 0

    def calculate_salary(self):
        """Рассчитывает оклад на основе отработанных часов."""
        self.salary = self.hourly_wage * self.hours_worked
        return self.salary

    def calculate_bonus(self):
        """Рассчитывает премию в зависимости от стажа."""
        if self.experience < 1:
            percent = 0.01
        elif self.experience < 3:
            percent = 0.05
        elif self.experience < 5:
            percent = 0.08
        else:
            percent = 0.15

        self.bonus = self.salary * percent
        return self.bonus

    def display_info(self):
        """Выводит полную информацию о сотруднике."""
        total_pay = self.salary + self.bonus
        print("-" * 30)
        print(f"Сотрудник: {self.fio}")
        print(f"Стаж: {self.experience} лет")
        print(f"Отработано часов: {self.hours_worked}")
        print(f"Оклад: {self.salary:.2f} руб.")
        print(f"Премия: {self.bonus:.2f} руб.")
        print(f"ИТОГО К ВЫПЛАТЕ: {total_pay:.2f} руб.")

employees_list = []

try:
    count = int(input("Сколько сотрудников вы хотите добавить? "))

    for i in range(count):
```

```
        print(f"\nВвод данных для сотрудника №{i+1}:")
        name = input("Введите ФИО: ")
        exp = float(input("Введите стаж (лет): "))
        wage = float(input("Введите часовую ставку: "))
        hours = float(input("Введите количество отработанных часов: "))

        emp = Employee(name, exp, wage, hours)

        emp.calculate_salary()
        emp.calculate_bonus()

        employees_list.append(emp)

    print("\n" + "="*10 + " СПИСОК СОТРУДНИКОВ " + "="*10)
    for emp in employees_list:
        emp.display_info()

except ValueError:
    print("Ошибка! Пожалуйста, вводите числовые значения для стажа, ставки и часов.")

# Блок 3, Задание 3
class Animal:
    def __init__(self, breed, price):
        self.breed = breed
        self.price = price

    def movement_method(self):
        return "Передвигается как-то"

    def __str__(self):
        return f"Порода: {self.breed}, Цена: {self.price}, Движение:
{self.movement_method()}"

class Fish(Animal):
    def movement_method(self):
        return "Плавает в воде"

class Birds(Animal):
    def movement_method(self):
        return "Летает в небе"

class PetShop:
    def __init__(self, name):
        self.name = name
        self.animals = []

    def add_animal(self, animal):
        self.animals.append(animal)

    def get_most_expensive(self):
        if not self.animals:
            return None
        return max(self.animals, key=lambda a: a.price)

    def save_to_file(self, filename="zoo_inventory.txt"):
        try:
            with open(filename, "w", encoding="utf-8") as f:
                f.write(f"Инвентаризация магазина '{self.name}':\n")
```

```
                f.write("-" * 40 + "\n")
                for a in self.animals:
                    f.write(str(a) + "\n")
            print(f"Информация успешно записана в файл: {filename}")
        except Exception as e:
            print(f"Ошибка при записи в файл: {e}")

shop = PetShop("Любимый питомец")

shop.add_animal(Fish("Золотая рыбка", 500))
shop.add_animal(Fish("Акула", 50000))
shop.add_animal(Birds("Волнистый попугай", 1500))
shop.add_animal(Birds("Орёл", 25000))

expensive = shop.get_most_expensive()
if expensive:
    print(f"Самая дорогая порода: {expensive.breed} (Цена: {expensive.price})")

shop.save_to_file()

print("\nСписок всех животных:")
for item in shop.animals:
    print(item)

# Блок 3, Задание 4
class Smartphone:
    brand_count = 0

    def __init__(self, model, brand, battery=100):
        self.model = model
        self.brand = brand
        self.battery = battery
        Smartphone.brand_count += 1

    def make_call(self, number):
        if self.battery > 5:
            self.battery -= 5
            print(f"Звонок на номер {number} с {self.model}... (Заряд:
{self.battery}%)")
        else:
            print("Слишком низкий заряд батареи!")

    @classmethod
    def create_iphone(cls, model):
        print(f"Создаем новый iPhone: {model}")
        return cls(model, "Apple")

    @classmethod
    def get_total_phones(cls):
        return f"Всего выпущено смартфонов: {cls.brand_count}"

    @staticmethod
    def is_valid_number(number):
        return number.startswith('+') and len(number) > 10

print(f"Проверка номера +79991234567: {Smartphone.is_valid_number('+79991234567')}")

phone1 = Smartphone.create_iphone("15 Pro")
```

```
phone1.make_call("+79001112233")

phone2 = Smartphone("Galaxy S23", "Samsung", battery=10)
phone2.make_call("+79995554433")

print(Smartphone.get_total_phones())
```

5 Скриншоты выполнения

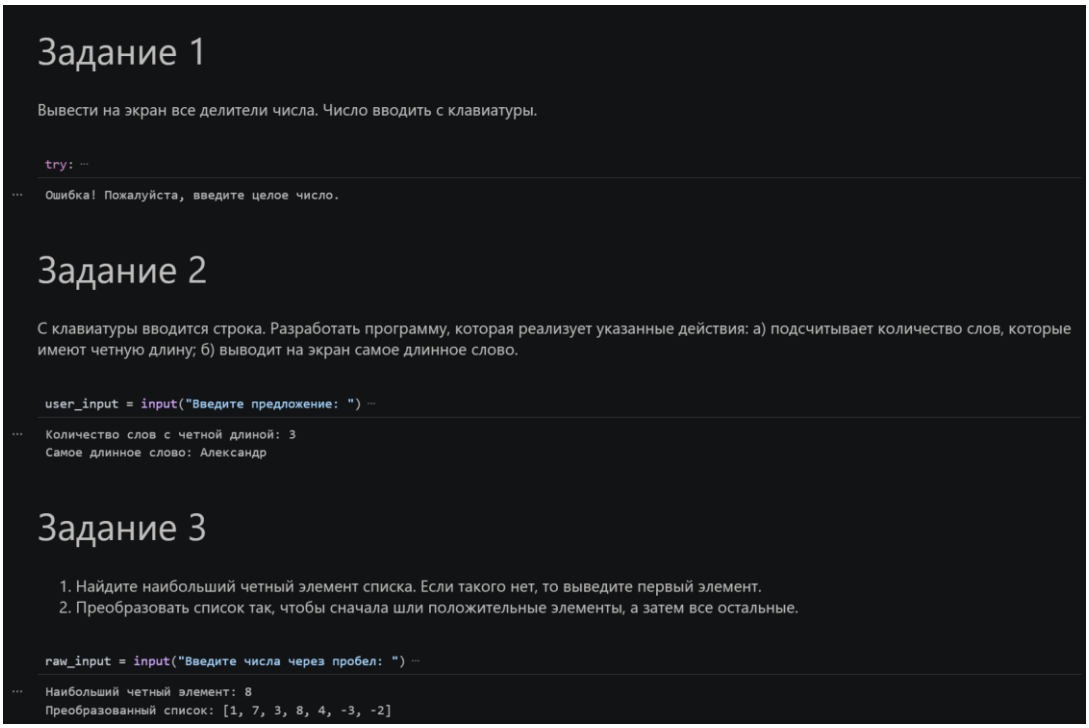


Рисунок 1 – Результаты выполнения заданий 1-3 блока 1

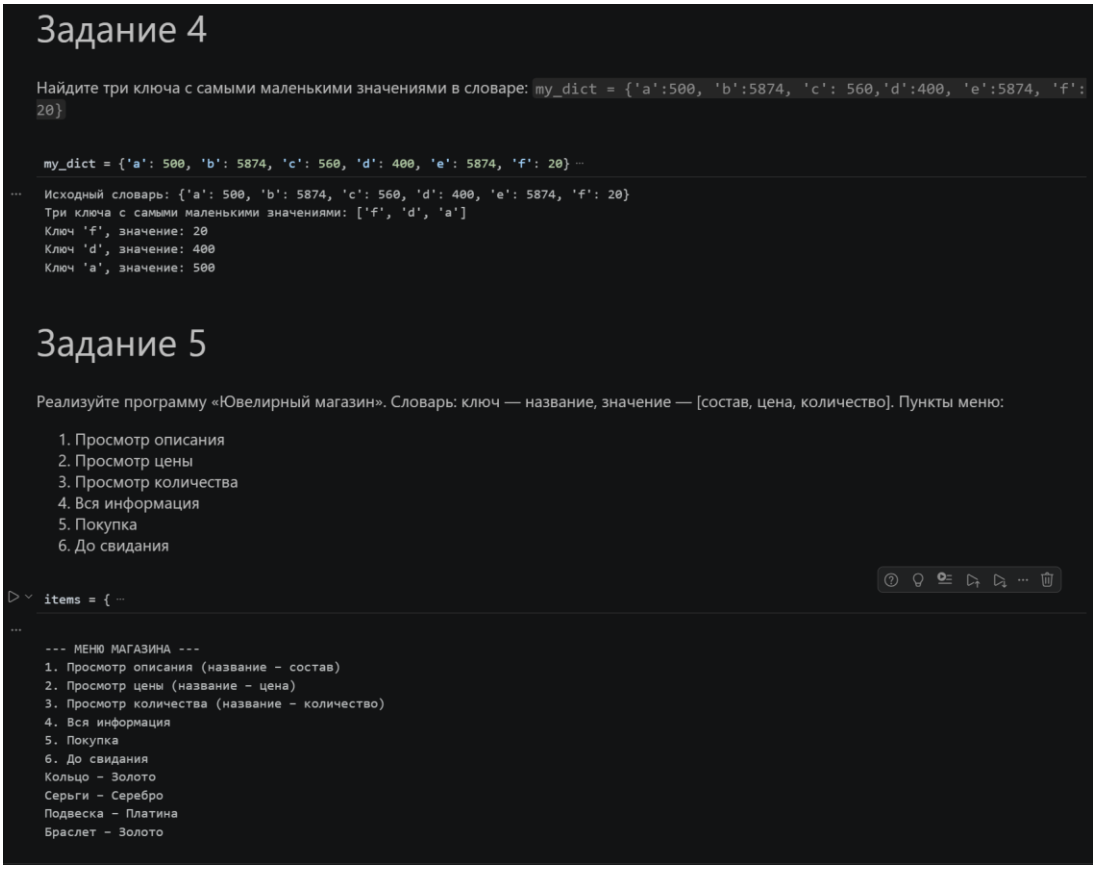


Рисунок 2 – Результаты выполнения заданий 4-5 блока 1

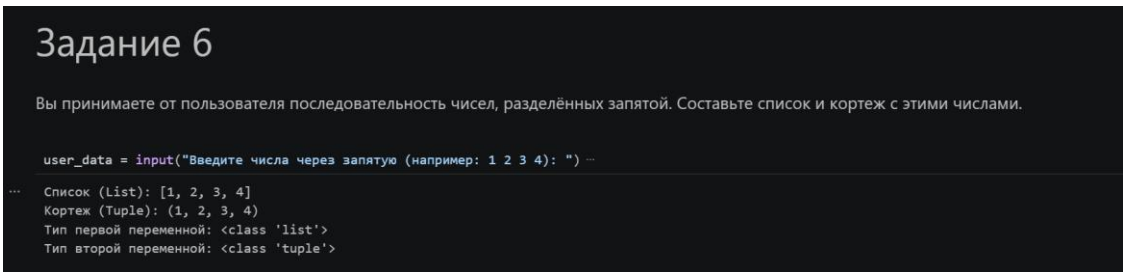


Рисунок 3 – Результаты выполнения задания 6 блока 1

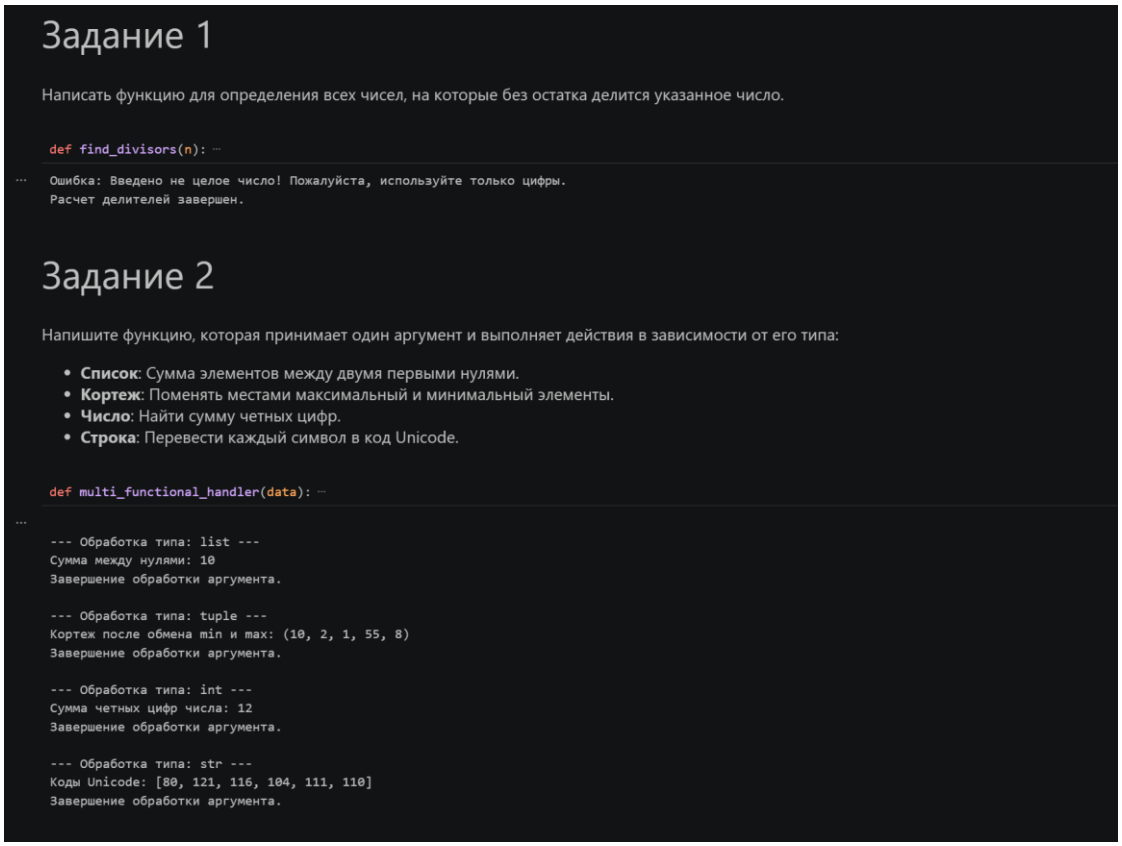


Рисунок 4 – Результаты выполнения заданий 1-2 блока 2

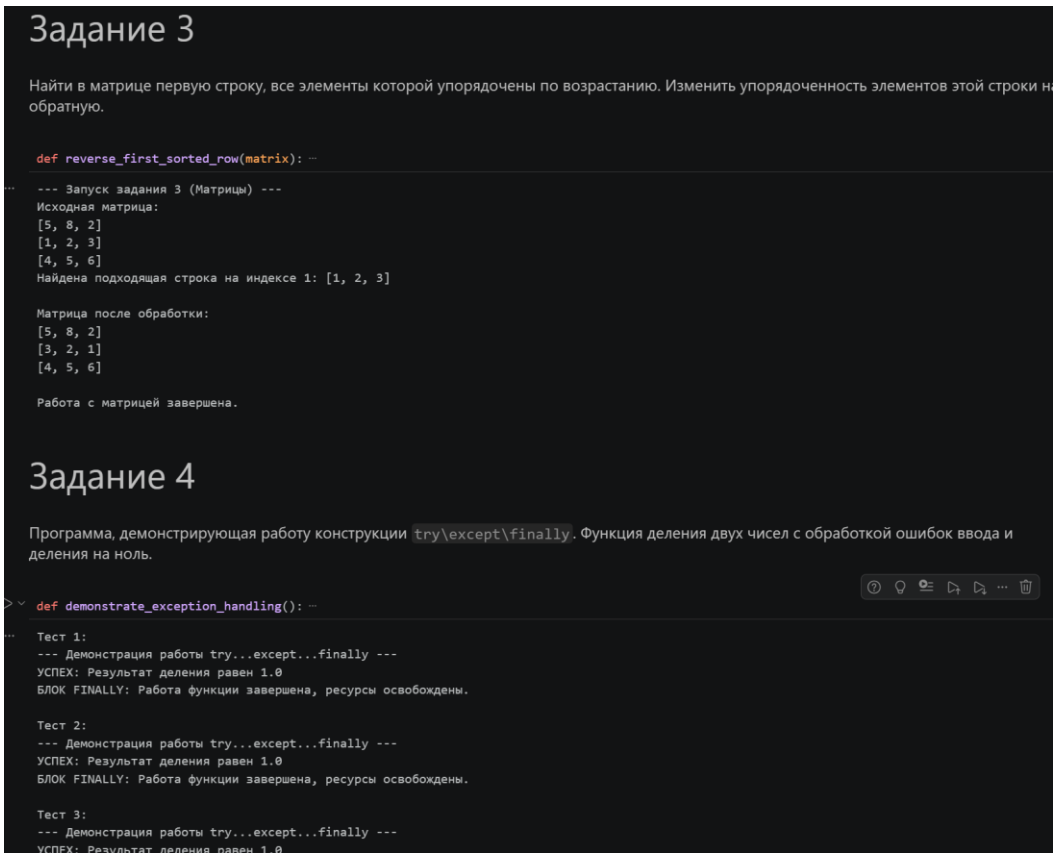


Рисунок 5 – Результаты выполнения заданий 3-4 блока 2

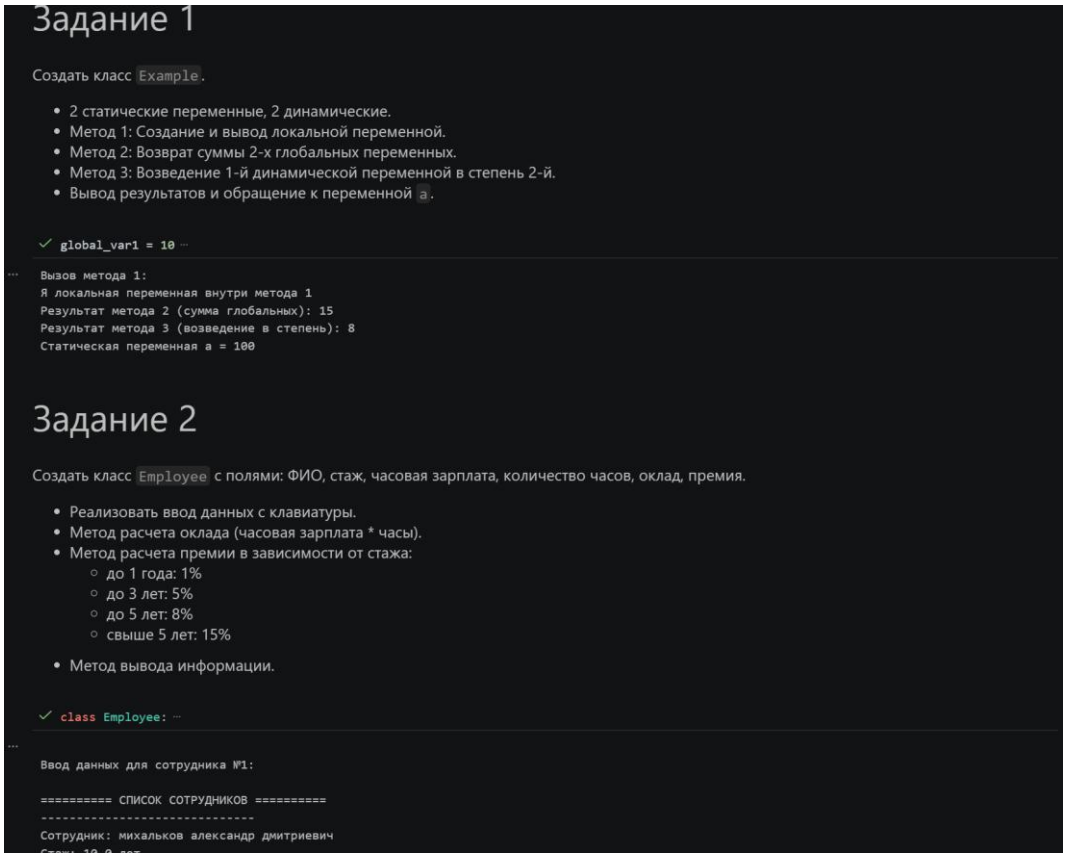


Рисунок 6 – Результаты выполнения заданий 1-2 блока 3

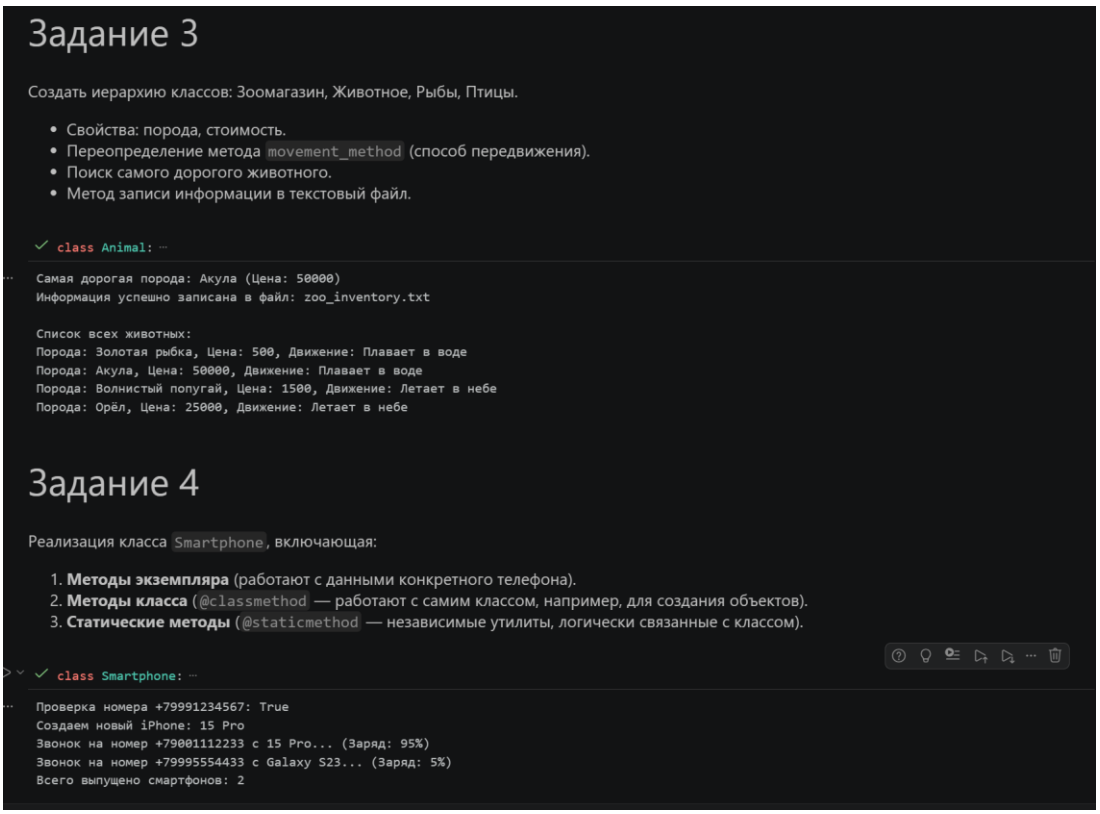


Рисунок 7 – Результаты выполнения заданий 3-4 блока 3

6 Выводы

В ходе выполнения лабораторной работы были изучены основы синтаксиса языка программирования Python и получены навыки проектирования приложений с использованием процедурного и объектно-ориентированного подходов.

На основании полученных результатов можно сделать следующие выводы:

1 Работа со структурами данных: На практике реализованы алгоритмы обработки стандартных коллекций (списков, кортежей и словарей). Установлено, что использование словарей является наиболее эффективным способом организации данных с быстрым доступом по ключу (на примере задачи «Ювелирный магазин»), а кортежи обеспечивают целостность данных за счет своей неизменяемости.

2 Декомпозиция и повторное использование кода: Использование функций позволило структурировать программный код, разделив логику обработки данных и интерфейс взаимодействия с пользователем. Применение параметров и возвращаемых значений функций обеспечило универсальность разработанных модулей.

3 Отказоустойчивость ПО: Реализованы механизмы обработки исключительных ситуаций с использованием блоков try-except-else-finally. Это позволило исключить аварийное завершение программ при некорректном вводе данных пользователем (например, ввод строки вместо числа или деление на ноль), что является критически важным для разработки надежных приложений.

4 Освоены принципы создания классов и инициализации объектов через конструктор __init__.

5 Изучены различия между методами экземпляра, методами класса (@classmethod) и статическими методами (@staticmethod), что позволило эффективно распределить логику работы между конкретными объектами и глобальной структурой класса.

6 На примере иерархии «Животное – Птицы – Рыбы» реализован механизм наследования и полиморфизма (переопределение методов передвижения), что позволило сократить дублирование кода.

7 Взаимодействие с внешними ресурсами: Реализован экспорт данных из программы в текстовый файл с корректным управлением ресурсами через менеджер контекста with, что обеспечило сохранность информации после завершения работы программы.

Таким образом, цель лабораторной работы достигнута, все поставленные задачи выполнены в полном объеме.